Practical course

Systems Programming

Systems Research Group

Today’s topic!

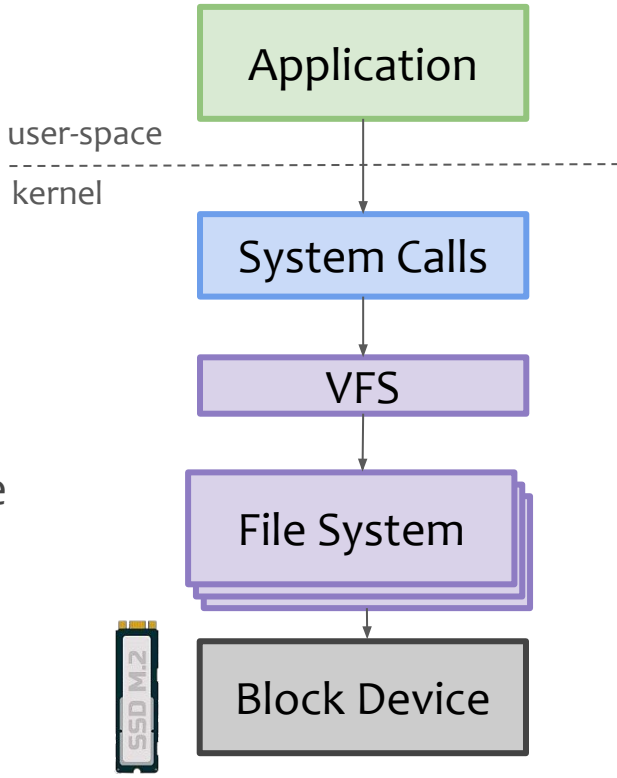
File I/O

Outline

- What is a filesystem?
- Filesystem abstractions
- Filesystem interface
- A very simple file system
- Read/Write on a very simple file system
- Speeding up filesystem operations

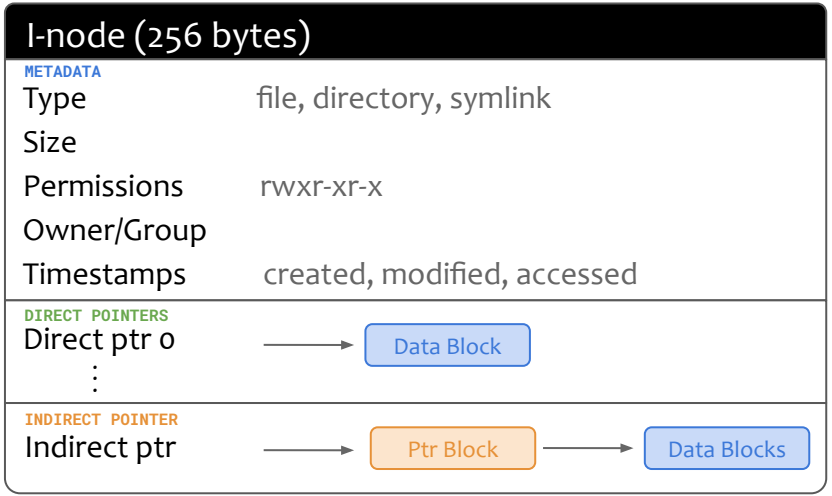
What is a filesystem?

- UNIX's defining feature: "Everything is a file"
- Network sockets, devices, data on disk...
- Filesystem is an implementation of the file interface
  - Decide how data is stored on disk
  - Performance, scalability, failure handling etc
  - Common operations include open/close, read/write
- Popular examples
  - *Ext4, NTFS, FAT32*
  - *procfs, sysfs*
  - *tmpfs*



Filesystem abstractions: File

- A file is a linear array of bytes
- Each file has a low-level identifier: the i-node number
- The i-node stores metadata (size, permissions, timestamps, type) and pointers to data blocks
- Metadata accesses via `stat`, files created via `open`



Filesystem abstractions: Directory

- Each directory has an i-node number

- Contents are specific
  - Mapping between i-nodes and names
  - Eg: `(foo,10)`

- Directory hierarchy.
  - Root directory (/)
  - "/" used to separate subsequent subdirectories and files
  - Eg: `/foo/bar/hello.txt`

Filesystem interface

- Create files or directories
- Access files or directories
  - Read or write
- Delete files or directories

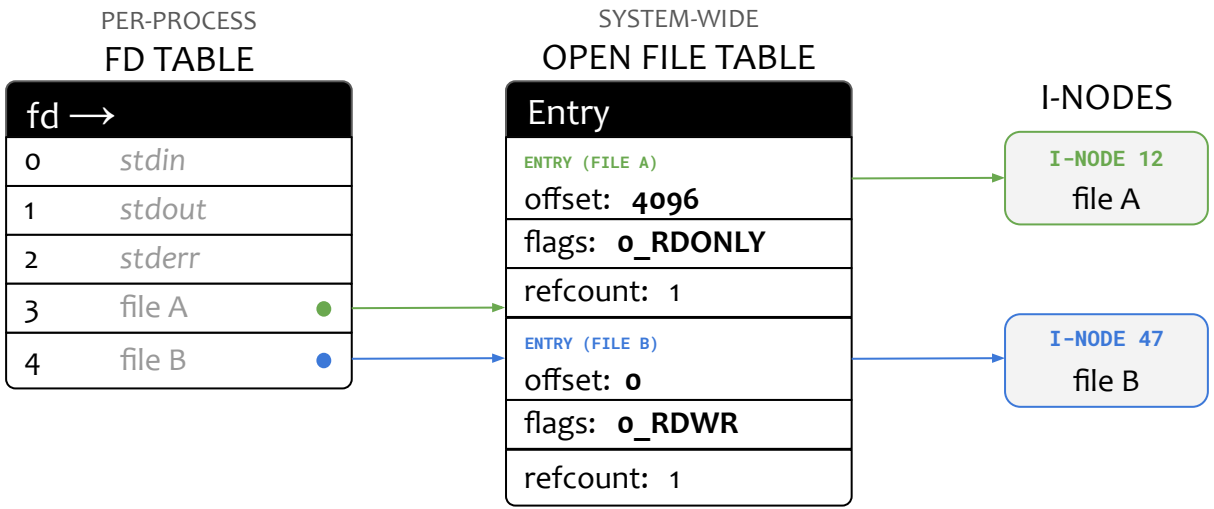
Filesystem interface: Detour

- strace traces all system calls made by a program

```
prompt> strace cat foo
...
openat(AT_FDCWD, "foo", O_RDONLY)      = 3          ← open file, get fd 3
fstat(3, {st_size=6, ...})             = 0          ← get file metadata
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0       ← hint: sequential read
mmap(NULL, 139264, ...)                = 0x7f...    ← allocate read buffer
read(3, "hello\n", 131072)             = 6          ← read 6 bytes from fd 3
write(1, "hello\n", 6)                 = 6          ← write to stdout (fd 1)
read(3, "", 131072)                                 ← EOF, nothing left
...
```

Filesystem interface: File Descriptors

- `int open(const char *pathname, int flags, mode_t mode)`
- Returns a **file descriptor (fd)** - a per-process integer
- fd indexes the per-process FD table → system-wide open file table → i-node
- Every process has 3 fds: stdin(0), stdout(1), stderr(2).



Filesystem interface: Core Operations

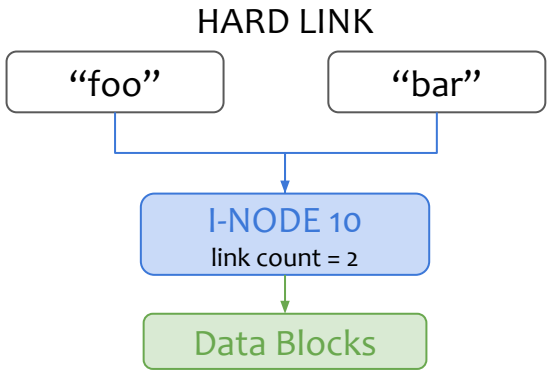
- `ssize_t read(int fd, void *buf, size_t count)`
  - Return number of bytes read
- `ssize_t write(int fd, const void *buf, size_t count)`
  - Return number of bytes actually written
- `off_t lseek(int fd, off_t offset, int whence)`
  - Change read or write offset into the file
  - Tracked by the OS for each open file.
- `int close(int fd)`
  - Releases the file descriptor

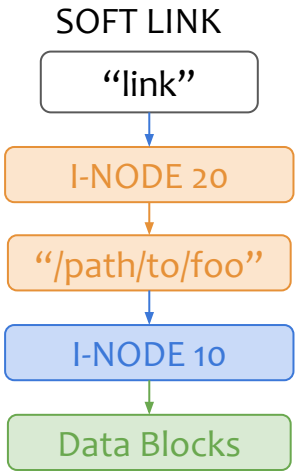
Filesystem interface: Other Operations

- Files: `stat()` - get metadata, `rename()`, `unlink()` - remove a file
- Directories: `mkdir()`, `readdir()`, `rmdir()`
- Persistence: `fsync(int fd)` - force buffered writes to disk

Filesystem interface: Hard and soft links

- Hard links: `ln [TARGET] [LINK]` - another name for the same i-node
  - File deleted only if number of links is 0
  - Cannot link across partitions or directories
- Soft links: `ln -s [TARGET] [LINK]` - separate file storing target's path
  - Can dangle if target is deleted

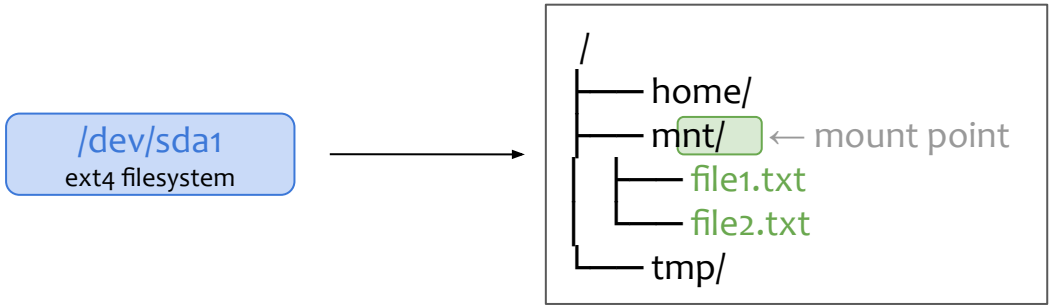




Filesystem interface: Make and mount

- **mkfs** - write an empty file system onto that partition
  - `mkfs -t ext4 /dev/sda1`

- **mount** - attaches a file system to a directory (mount point)
  - `mount /dev/sda1 /mnt`



A very simple file system: Layout

- 8 blocks, each 4 KiB. I-nodes are 256 bytes (16 per block).
- Disk layout:

- Superblock: filesystem metadata, read first at mount
- Bitmaps: track which i-nodes/data blocks are free
- I-node block: stores 16 i-nodes
- Data blocks: 4 blocks for file/directory contents

A very simple file system: I-node

- 16 i-nodes first per 4KiB block (4096/256 = 16)
- To find i-node N on disk:
  - `block = inode_start + (N / inodes_per_block)`
  - `offset = (N % inodes_per_block) × 256`
- Example: i-node 12 ⟶ block 3, offset 3072

I-NODE BLOCK (BLOCK 3)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	256	512	768	1024	1280	1536	1792	2048	2304	2560	2816	3072	3328	3584	3840

I-node 12⟶ offset 3072

A very simple file system: Directory

- A directory is just an i-node with type = “directory”
- Its data blocks contain name → i-node mappings:

DIRECTORY I-NODE

I-node 2	
type:	directory
size:	4096
ptr 0:	block 4

DATA BLOCK 4 (DIRECTORY CONTENT)

Name	I-node
.	2
..	2
foo	12
bar	13

A very simple file system: Free space management

- Bitmaps track which i-nodes/data blocks are in use (1 = used, 0 = free)
- I-node bitmap tracks free inodes
- Data bitmap tracks free data blocks

I-NODE BITMAP

1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

DATA BITMAP

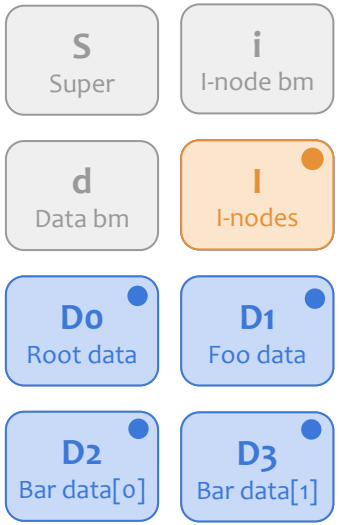
1	1	0	0
0	3	4	15

A very simple file system: Read operation

- Open `/foo/bar` and read 8 KiB (2 data blocks)
- The FS must traverse the directory tree to find the file:

A very simple file system: Read operation

#	SYSCALL	R/W	WHAT	WHY
`OPEN("/FOO/BAR")`	Traverse directory tree to find file			
1	**open**	R	Root i-node	Get root directory data block pointer
2	**open**	R	Root data	Look up “foo” → i-node 12
3	**open**	R	Foo i-node	Get foo directory data block pointer
4	**open**	R	Foo data	Look up “bar” → i-node 13
5	**open**	R	Bar i-node	Found the file, return fd
`READ() – BLOCK 0`	Read first 4 KiB of file			
6	**read**	R	Bar i-node	Get data block 0 address
7	**read**	R	Bar data[0]	Read first 4 KiB of file data
8	**read**	W	Bar i-node	Update last accessed data
`READ() – BLOCK 1`	Read next 4 KiB of file			
9	**read**	R	Bar i-node	Get data block 1 address
10	**read**	R	Bar data[1]	Read next 4 KiB of file data
11	**read**	W	Bar i-node	Update last accessed time

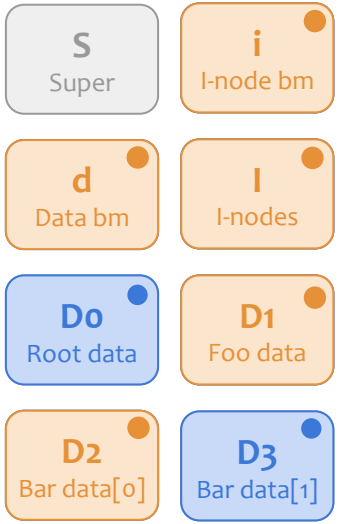


A very simple file system: Write operation

- Create `/foo/bar` and write 4KiB (1 data block)
- Writing requires allocating new inodes and data blocks:

A very simple file system: Write operation

#	SYSCALL	R/W	WHAT	WHY
`CREATE("/FOO/BAR")`	Traverse tree, allocate inode, update parent			
1	**create**	R	Root inode	Get root directory data block pointer
2	**create**	R	Root data	Look up “foo” → inode 12
3	**create**	R	Foo inode	Get foo directory data block pointer
4	**create**	R	Foo data	Check “bar” doesn’t already exist
5	**create**	R	Inode bitmap	Find a free inode for “bar”
6	**create**	W	Inode bitmap	Mark inode as allocated
7	**create**	W	Bar inode	Initialize new inode (type, size, etc.)
8	**create**	R	Foo data	Read parent dir to prepare update
9	**create**	W	Foo data	Add (“bar”, inode N) entry to parent dir
10	**create**	W	Foo inode	Update parent dir metadata (size, mtime)
`WRITE() – 4 KIB`	Allocate data block, write data, update inode			
11	**write**	R	Data bitmap	Find a free data block
12	**write**	W	Data bitmap	Mark data block as allocated
13	**write**	W	Bar data[0]	Write actual file data to disk
14	**write**	W	Bar inode	Update size, block pointer, mtime



Speeding Up: Caching & Write Buffering

- Problem: too many disk I/Os per operation (14 for a single write!)
- Read caching: keep recently read blocks in memory (page cache)
  - Repeated reads to same inode/data blocks → served from memory
  - Eviction policy: LRU
- Write buffering: batch writes in memory before flushing to disk
  - Fewer I/Os: combine multiple updates
  - Better scheduling: sequential writes faster than random
  - Avoid writes: file created and deleted before flush → no disk I/O
- **Tradeoff:** buffered writes lost on crash → `fsync(fd)` forces flush
  - Modern OSes use a **unified page cache** for both VM pages and file data

Thank you for listening!

See you in the Q&A session

A very simple file system: Organization

- 4 data blocks
- 1 inode block
- 1 block to store inode bitmaps
- 1 block to store data bitmaps
- 1 block to store the superblock of the filesystem
  - Contains filesystem metadata
  - Location is known
  - OS first reads this block before mounting the file system.

A very simple file system: Read operation

	Data bitmap	Inode bitmap	Root inode	Foo inode	Bar inode	Root data	Foo data	Bar data[0]	Bar data[1]
open(bar)			read	read	read	read	read		
read()					read write			read	
read()					read write				read

A very simple file system: Write operation

	Data bitmap	Inode bitmap	Root inode	Foo inode	Bar inode	Root data	Foo data	Bar data[0]
create(/foo/bar)			read			read		
				read			read	
		read						
		write					write	
					read			
					write			
				write				
write()	read				read			
	write							write
					write			

A very simple file system: Caching

- Poor performance
  - Request same data blocks from disk repeatedly
  - Slow operation compared to reading from memory

- Solution
  - Cache blocks in main memory
  - Strategies to decide which blocks to keep and which to evict from memory
    - LRU
  - Modern OSes integrate virtual memory pages and file system pages into a unified page cache.

A very simple file system: Caching

- Write buffering
  - Buffer writes in main memory instead of immediately persisting them.
  - Advantages:
    - Batch updates
    - Better scheduling of I/O
      - Sequential faster than random
    - Possibility of avoiding writes